CardStack

Design D2: Detailed Design

Goal Statement

CardStack is a payment system that allows consumers to complete online purchases quickly while automatically selecting the user's credit card that provides the best rewards for the purchase category.

1. Header, Conventions, and D2 Overview

Conventions

These conventions apply to every diagram in this document and are repeated under each one.

- **Rounded rectangles** represent CardStack components developed by the team.
- **Gray boxes** represent external systems that CardStack depends on (Google sign-in, Plaid, the payment processor, the merchant site).
- **Solid arrows** represent data or control being sent between components, in the direction of the arrow.
- **Dotted arrows** represent error or fallback paths.
- **Interface IDs (I1, I2, etc.)** identify the interfaces described in the Interface Table in Section 2. Error IDs (E-AUTH, E-SEL) are defined in Section 3.
- **State machines:** each box is a state, each arrow is a transition labeled with its trigger, and the filled circle and double circle mark the start and end.

D2 Overview

What the system is. CardStack adds a checkout button to third-party merchant sites. When a shopper clicks it, they sign in, CardStack picks the connected credit card with the best rewards for that purchase category, charges it through a payment processor, and reports the result to the merchant. Users manage their cards and see points earned in a mobile and a web app.

What D1 covered. The checkout workflow, the Checkout Button entity, two core algorithms (card processing and financial institution setup), build versus reuse, two API contracts (`/process-card-payment` and `/connect-bank-account`), and technology choices (React Vite, Go, Supabase, Google Cloud, Firebase). D1 deferred the mobile and web apps to D2.

What D2 adds. A data model, behavior, and interface for every remaining component (login and checkout pages, card selection, merchant webhook, transaction history, receipt forwarding, mobile app, web app) and for the glue components (authentication, notifications, configuration,

logging). It also adds an edge-case and error-handling table, a requirements traceability table, a design rationale, and the peer design review notes.

Known Gaps and Open Questions

- The virtual debit card (phase 2) is still in concept testing. It is out of scope for D2 and has no requirement behind it yet.
- Card-selection tie-breaking and how miles are compared with cash back are not decided. D1 lists them only as edge cases. See Automatic Card Selection.

2. Finished Components (D2 Diagrams)

System Diagram with Error Paths

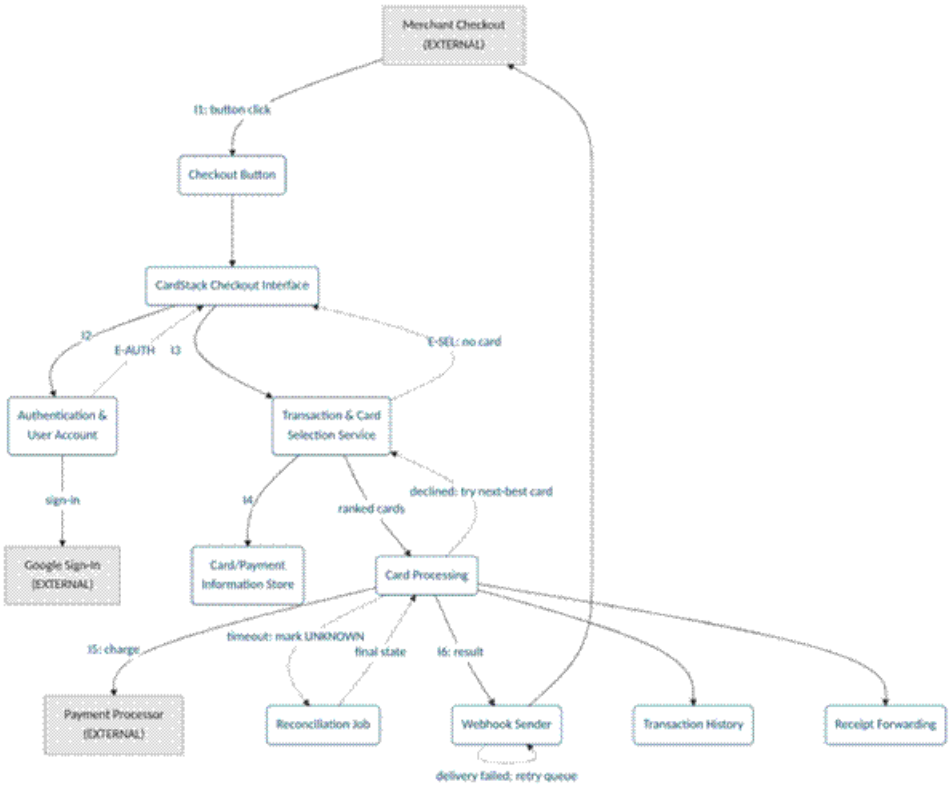


Figure 1. Checkout flow with error and fallback paths. Conventions as in Section 1: rounded rectangles are CardStack components, gray boxes are external systems, solid arrows are the main flow, dotted arrows are error paths.

Interface Table

I1 through I6 below follow the numbering in the D1 diagram; I7 through I11 are new in D2.

ID	From / To	Contract	Errors
I1	Merchant checkout, Checkout Button, Checkout Interface	Button click carrying button id, merchant origin, cart total and category	`ButtonNotAllowed`
I2	Checkout Interface to Auth & User Account	Google ID token in, session out	`AuthInvalidToken`, `AuthExpired`
I3	Checkout Interface to Card Selection Service	`POST /server/v1/select-card`, ranked card list out	`E-SEL-NO-CARD`
I4	Card Selection to Card/Payment Information Store	Read active cards and reward rules; write `selection_log`	`StoreUnavailable`
I5	Card Processing to Payment Processor	Charge using the processor token and an idempotency key (transactionId)	Declined, timeout
I6	Card Processing to Webhook Sender to Merchant	Signed result payload (transactionId, state, amount)	Delivery failure, retried
I7	Mobile and web apps to Auth	Same as I2, plus logout	`AuthInvalidToken`, `AuthExpired`
I8	Mobile and web apps to card management	List, add, remove cards and set preferred card; add through Plaid Link	`AddInstitutionError`
I9	Plaid to `/connect-bank-account`	Webhook after a link attempt (D1)	Invalid signature (EC-05)
I10	Mobile and web apps to Transaction History	`GET /server/v1/transactions?cursor=`	`AuthExpired`
I11	Card Processing to Receipt Forwarding and Notifications	Internal queue message on a terminal state	Queue retry

Automatic Card Selection

Data model

Reads `user_card` and `reward_rule`. Writes `selection_log` (transaction_id, chosen_card_id, rule_id, reason).

Behavior

1. Load the user's active cards and each card's reward rule for the transaction category.
2. Drop cards that are expired, frozen, or without enough available credit for the amount.
3. Score each remaining card: `effective_value = amount x rate x point_value_cents`. The point value is what makes miles and cash back comparable.
4. The highest score wins. Tie-break: the user's preferred card, then lowest utilization, then oldest card.
5. If the user has only one card, select it without scoring.
6. Return an ordered ranked list, not only the winner, so Card Processing can fall back to the next card if one is declined.

Interface

`POST /server/v1/select-card`. In: userId, transactionId, amount, category, merchant. Out: ranked list of `{cardId, expectedValue}`, or an error.

Authentication & User Account

Data model

`user` (id, email, google_sub, created_at, status) and `session` (token_hash, user_id, expires_at).

Behavior

State machine: ANONYMOUS, SIGNING_IN, AUTHENTICATED, EXPIRED. Sessions are short-lived, and an expired session triggers a new login without losing the cart.

Interface

`POST /server/v1/auth/google` (ID token in, session out) and `POST /server/v1/auth/logout`. Errors: `AuthInvalidToken`, `AuthExpired`.

Note: D1 gives userId two formats (a UUID in one contract, a `cardstack-user-` prefix in another). Pick one in the interface revisions below.

Card/Payment Information Store

Relational model, as chosen in D1. The store holds processor tokens and the last four digits only, never full card numbers.

Table	Key fields	Notes
`user`	id, email, google_sub	See Authentication
`institution_link`	id, user_id, plaid_item_id, status	Written from the Plaid webhook

`user_card`	id, user_id, institution_link_id, processor_token, last4, network, preferred, active	Tokens and last4 only
`reward_rule`	id, card_product_id, category, rate, reward_type	Researched per the D1 build decision
`transaction`	id (UUID), user_id, card_id, merchant_id, amount_cents, category, state, error_code, created_at	State per the lifecycle below
`checkout_button`	button_id, merchant_id, allowed_domain	Indexed, per D1
`webhook_delivery`	id, transaction_id, attempts, next_retry_at, status	Drives retries
`selection_log`	transaction_id, chosen_card_id, rule_id, reason	Audit trail

Amounts are stored as integer cents. D1's contract shows a Float64 amount, which risks rounding errors on money.

Transaction Lifecycle

CardStack. Goal statement: CardStack is a payment system that allows consumers to complete online purchases quickly while automatically selecting the user's credit card that provides the best rewards for the purchase category.

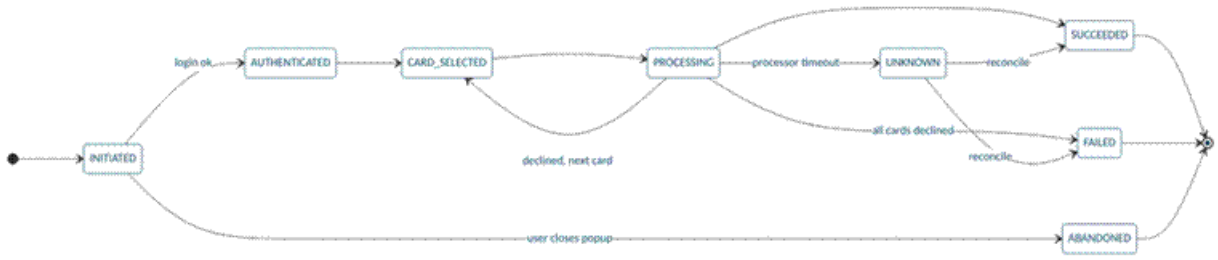


Figure 2. Transaction state machine. Conventions as in Section 1: each box is a state and each arrow is a transition labeled with its trigger. PROCESSING returns to CARD_SELECTED when a card is declined and another is available; UNKNOWN is resolved by the reconciliation job.

Remaining Components

Component	Data model	Behavior	Interface
Checkout Button	`checkout_button` (D1)	Loads on the merchant page, validates button id and origin against `allowed_domain`, opens the popup	`GET /server/v1/button/{id}`, returns config or `ButtonNotAllowed`
Login popup and checkout pages	No persistent data (session only)	Sign in, show cart and chosen card, confirm, show result	Calls I2 and I3

Card Processing	Updates `transaction`	D1 algorithm plus: idempotency key = transactionId; on a decline, request the next ranked card; on a timeout, set UNKNOWN	`/process-card-payment` (D1, revised below)
Financial Institution Setup	`institution_link`, `user_card`	Plaid Link completes, webhook arrives, verify signature, store tokens, redirect	`/connect-bank-account` (D1)
Merchant webhook	`webhook_delivery`	Send a signed payload, retry with backoff, stop after N attempts and flag the merchant	I6: `POST merchant_url` with an HMAC header
Transaction history	`transaction`	Append on a terminal state; read by the apps, paginated	`GET /server/v1/transactions?cursor=`
Receipt forwarding	`receipt` (transaction_id, email, status)	On SUCCEEDED, email a receipt to the user	Internal queue message (I11)
Mobile app (iOS/Android)	Local cache only	Screens: sign-in, cards, add card (Plaid Link), points, history	I7, I8, I10
Web app	No persistent data	Same functions as the mobile app; extra settings later, per D1	I7, I8, I10
Notifications (glue)	`notification` (user_id, type, sent_at)	Push or email for payment results and link failures	Internal (I11)
Configuration (glue)	No persistent business data (secret manager, environment)	Plaid, processor and webhook keys kept outside the repo; feature flags	None
Logging (glue)	Log store; never log card tokens or full personal data	Structured logs correlated by transactionId	Internal
Virtual debit card	None	Out of scope for D2	None

Interface Revisions based on D1

- `/process-card-payment` takes `cardId` from the caller, but selection now returns a ranked list. Better to have server call selection internally, so the frontend sends only transactionId, amount and category.
- Use one userId format across all contracts.
- `amount`: integer cents, not Float64.

- Use 400 bad input, 401 auth, 409 duplicate, 502 processor failure and error code instead of HTTP 500 for all errores.
- No raw card data will be passed through CardStack servers (PCI scope), only processor and Plaid tokens will be used.

3. Edge Cases and Error Handling

ID	Component	Category	Edge case	Decision	Handler	Recov.
EC-01	Card Selection	Boundary	User has exactly one card	Skip scoring, select it	Selection Service	Yes
EC-02	Card Selection	Boundary	Two cards tie on value	Apply the tie-break order	Selection Service	Yes
EC-03	Card Processing	Timing	Double-click sends the same transaction twice	Idempotency on transactionId; return the first result	Card Processing	Yes
EC-04	Card Processing	External	Processor times out	Mark UNKNOWN; reconciliation job queries the processor	Reconciliation Job	Yes
EC-05	Financial Institution Setup	Bad input	Plaid webhook has an invalid signature	Reject, log, store nothing	Setup endpoint	Yes (user retries)
EC-06	Merchant webhook	External	Merchant endpoint is down	Exponential backoff, N attempts, then flag	Webhook Sender	Partly
EC-07	Checkout Button	Bad input	Invalid login credentials	Prompt user up to 5 times to retry, then suggest account recovery	Login popup	yes
EC-08	Auth	Timing	Session expires mid-checkout	Scraps session, logs out user	Login popup	no
EC-09	Mobile/web apps	boundary	User has no cards	Show empty state with add a card plaid link	App client	yes

4. Requirements Traceability

Req ID	Requirement	Priority	Design elements	How

R-1	Shopper checks out with one click on a merchant site	High	`checkout_button` entity, `GET /button/{id}`, Figure 1	Id of the user is required to pull the financial data
R-2	System picks the best-rewards card per category	High	Selection algorithm, `/reward_rule/{id}`, EC-01, EC-02	Id of the user will be used to get credit card reward information to charge
R-3	User connects cards and bank accounts	High	`/connect-bank-account`, `institution_link`, EC-05	Plaid will hit our webhook with the accountId to store in our database
R-4	User views points and the card used per transaction	Medium	`transaction`, `selection_log`, history endpoint (I10)	UserId needed to query the database. Send the json back to the frontend
R-5	Card data is secure	High	Token-only storage, configuration secrets, logging rules	Store encrypted accountIds from plaid in the database
R-6	Checkout is fast	Medium	Indexed `button_id`, single ranked-list call	Once the checkout button is clicked needs to be under 5 seconds for the transaction to go through to the correct card

5. Design Rationale

1. We chose a relational database (Supabase) over a document store because a database allows us to query any sort of data with Golangs built in sql variables, making it easier to format the data right from the sql call. We accepted the trade-off of not having JSON formatted data with a document store because the sql database offers strict guarantees of and consistency with the data. There are foreign key constraints that can lower the amount of dead data in the sql servers.
2. We chose using Plaid for bank connection over building our own because Plaid has connections to all bank institutions with a dedicated team for support. We accepted the trade-off of using Plaid because Plaid is a trusted third party api source used by many top companies like Credit Karma. Consumers also trust Plaid to enter their bank credentials. In contrary, making our own would be forcing the consumer to put the trust in us with their bank credentials which many may not.
3. We chose reusing a payment processor over handling charges ourselves because building our own payment processor would take a lot of time and a full dedicated team to set up. We

accepted the trade-off of paying a fee per transaction because of the support the api processor offers and the trade-off of not needing to build out the algorithm.

4. We chose a Go backend over Node because the language is widely known for api's that we are using. For example, Plaid has a fully supported package for Golang instead of having to use direct HTTP calls. We accepted the trade-off of not using React Node with the frontend application React Native because Golang can act as a seperate server. We can make multiple instances of those servers in case one of the servers fails another can pick back up.
5. We chose React Native for web and mobile over React Vite because Native allows us to build out the web application and mobile application in one project instead of having two different codebases. We accepted the trade-off of a more complex project structure because of the cross reference of using the same components for web and mobile applications.
6. We chose using React error boundaries for specific components over global error handling because it allows individual components to fail instead of the entire page or website to crash for the user. The specific component can go under error retry while the rest of the page is working for the user.
7. We chose integer cents over floats for money because it gives more accurate data when combining numbers. We accepted the trade-off of floats because integer cents gives the exact value of the money amount. When it comes to floats there can be straggling numbers at the decimal level that can give inaccurate data. Its small but when dealing with money its crucial to have the exact amount.
8. We chose Microsoft Azure to host our web app over other third party hosters because Azure is tied with Github and allows us to push code directly to github and automatically update production. We accepted the trade-off of Azure because it is extremely secure in cyber attacks against our application.
9. We chose a webhook retry queue over fire-and-forget because some webhooks may rely on other webhooks calling data to work. We accepted the trade-off of webhook retry queue because a webhook couldve gotten caught in a dead spot with important information submitted by the user like a form, or even submitting institution data after they connected an account. If plaid registers a connection and we dont store it in the database it could result in losing the user secret.
10. We chose Microsoft Azure to host our backend servers over other third party hosters because Azure lets us dynamically scale our servers with traffic. We accepted the trade-off of Azure because it is cost effective and allows us to pay the minimal costs of hosting while starting off and upscale during high volumes of traffic.

6. Peer Design Review Notes

ID	Criterion	Issue (team)	Action	Owner	Priority	Due

PR-1	Feasibility	Plaid can connect bank and credit accounts for reading data, but it may not hand back a credit card token that can be charged.	Explain how the card can become chargeable.	Maya	Medium	11/10
PR-2	UX	IF a user's card is declined a different card is automatically charged	Add explicit choices a user needs to make for a different card to be charged.	Maya	Low	End of Semester
PR-3	Flow	Some of the diagrams (ex. Tech stack vs rationale) do not match.	Review all parts to make sure they are synthesized.	Luke	High	10/31